Research Notes on Proximal policy optimization

Proximal policy optimization

>> Section 1

Proximal policy optimization (PPO) is a reinforcement learning (RL) algorithm for training an intelligent agent. Specifically, it is a policy gradient method, often used for deep RL when the policy network is very large.

>> Section 2

If r t (θ) > 1 {\displaystyle r_{t}(\theta)>1} , where θ {\displaystyle \theta } are the policy network parameters, then selecting action a {\displaystyle a} in state s {\displaystyle s} is more likely based on the current policy than the previous policy. If 0 ≤ r t (θ) < 1 {\displaystyle 0\leq r_{t}(\theta)<1} , then selecting action a {\displaystyle a} in state s {\displaystyle s} is less likely based on the current policy than the old policy. Hence, this ratio function can easily estimate the divergence between old and current policies.

>> Section 3

for k = 0 , 1 , 2 , … {\textstyle k=0,1,2,\ldots } do Collect set of trajectories D k = { τ i } {\textstyle {\mathcal {D}}_{k}=\left\{\tau _{i}\right\}} by running policy π k = π (θ k) {\textstyle \pi _{k}=\pi \left(\theta _{k}\right)} in the environment. Compute rewards-to-go R ^ t {\textstyle {\hat {R}}_{t}} . Compute advantage estimates, A ^ t {\textstyle {\hat {A}}_{t}} (using any method of advantage estimation) based on the current value function V φ k {\textstyle V_{\phi _{k}}} . Update the policy by maximizing the PPO-Clip objective: θ k + 1 = arg ⁡ max ⁡ θ 1 | D k | T ∑ τ ∈ D k ∑ t = 0 T min (π θ (a t ■ s t) π θ k (a t ■ s t) A π θ k (s t , a t) , g (■ , A π θ k (s t , a t))) {\displaystyle \theta _{k+1}=\arg \max _{\theta }{\frac {1}{\left|{\mathcal {D}}_{k}\right|T}}\sum _{\tau \in {\mathcal {D}}_{k}}\sum _{t=0}^{T}\min \left({\frac {\pi _{\theta }\left(a_{t}\mid s_{t}\right)}{\pi _{\theta _{k}}\left(a_{t}\mid s_{t}\right)}}A^{\pi _{\theta _{k}}}\left(s_{t},a_{t}\right),\quad g\left(\epsilon ,A^{\pi _{\theta _{k}}}\left(s_{t},a_{t}\right)\right)\right)} typically via stochastic gradient ascent with Adam. Fit value function by regression on mean-squared error: φ k + 1 = arg ⁡ min ⁡ φ 1 | D k | T ∑ τ ∈ D k ∑ t = 0 T (V φ (s t) − R ^ t) 2 {\displaystyle \phi _{k+1}=\arg \min _{\phi }{\frac {1}{\left|{\mathcal {D}}_{k}\right|T}}\sum _{\tau \in {\mathcal {D}}_{k}}\sum _{t=0}^{T}\left(V_{\phi }\left(s_{t}\right)-{\hat {R}}_{t}\right)^{2}} typically via some gradient descent algorithm. Like all policy gradient methods, PPO is used for training an RL agent whose actions are determined by a differentiable policy function by gradient ascent. Intuitively, a policy gradient method takes small policy update steps, so the agent can reach higher and higher rewards in expectation. Policy gradient methods may be unstable: A step size that is too big may direct the policy in a suboptimal direction, thus having little possibility of recovery; a step size that is too small lowers the overall efficiency. To solve the instability, PPO implements a clip function that constrains the policy update of an agent from being too large, so that larger step sizes may be used without negatively affecting the gradient ascent process.

>> Section 4

Basic concepts To begin the PPO training process, the agent is set in an environment to perform actions based on its current input. In the early phase of training, the agent can freely explore solutions and keep track of the result. Later, with a certain amount of transition samples and policy updates, the agent will select an action to take by randomly sampling from the probability distribution P (A | S) {\displaystyle P(A|S)} generated by the policy network. The actions that are most likely to be beneficial will have the highest probability of being selected from the random sample. After an agent arrives at a different scenario (a new state) by acting, it is rewarded with a positive reward or a negative reward. The objective of an agent is to maximize the cumulative reward signal across sequences of states, known as episodes.

>> Section 5

Ratio function In PPO, the ratio function (r t {\displaystyle r_{t}}) calculates the probability of selecting action a {\displaystyle a} in state s {\displaystyle s} given the current policy network, divided by the previous probability under the old policy. In other words:

>> Section 6

clip ■ (r t (θ)) ⋅ A {\displaystyle \operatorname {clip} (r_{t}(\theta))\cdot A} : the policy ratio is first clipped to the range [1 − ■ , 1 + ■] {\displaystyle [1-\epsilon ,1+\epsilon]} ; generally, ■ {\displaystyle \epsilon } is defined to be 0.2. Then, as before, it is multiplied by the advantage. The fundamental intuition behind PPO is the same as that of TRPO: conservatism. Clipping results in a conservative advantage estimate of the new policy. The reasoning is that if an agent makes significant changes due to high advantage estimates, its policy update will be large and unstable, and may diverge from the optimal policy with little possibility of recovery. There are two common applications of the clipping function: when an action under a new policy happens to be a good choice based on the advantage function, the clipping function limits how much credit can be given to a new policy for up-weighted good actions. On the other hand, when an action under the old policy is judged to be bad, the clipping function constrains how much the agent can accept the down-weighted bad actions of the new policy. Consequently, the clipping mechanism is designed to discourage the incentive of moving beyond the

defined range by clipping both directions. The advantage of this method is that it can be optimized directly with gradient descent, as opposed to the strict KL divergence constraint of TRPO, making the implementation faster and more intuitive. After computing the clipped surrogate objective function, the agent has two probability ratios: one non-clipped and one clipped. Then, by taking the minimum of the two objectives, the final objective becomes a lower bound (a pessimistic bound) of what the agent knows is possible. In other words, the minimum method makes sure that the agent is doing the safest possible update.